

Maschinelles Lernen für die Chemie - Exercise 3

Logistic Regression, Support Vector Machines

Leonie Pfeffer, Maike Vogelbacher | May 21, 2025

Logistic Regression for Classification

Definition

- Supervised learning
- Predict categorical outcomes
 - Binary classification: yes/no, 1/0
 - Multiple classes

Goal

Predict the probability that a sample x belongs to class 1

$$P(y = 1 \mid x)$$

- $x \in \mathbb{R}$: a single input feature
- $y \in \{0, 1\}$: target class label

Decision Rule

Binary classification with decision boundary at 0.5

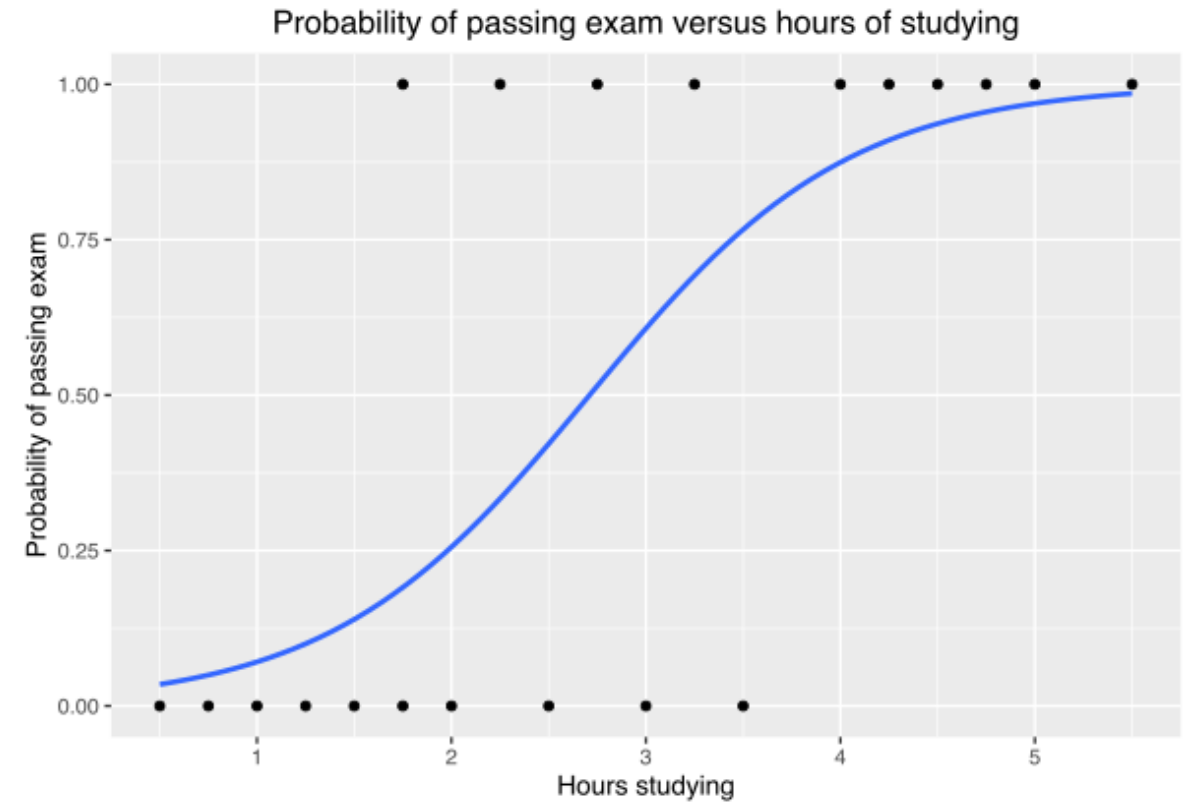
$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Sigmoid Function

a.k.a logistic function

$$P(y = 1 \mid x) = \frac{1}{1 + \exp[-(wx + b)]}$$

- w : weight (slope)
- b : bias (intercept)



Multi-class Classification: Multinomial Regression

Example: Iris Dataset

- $n = 150$ samples
- $d = 4$ features (sepal length/width, petal length/width)
- $K = 3$ classes (setosa, versicolor, virginica)
- $\mathbf{x}^{(i)} \in \mathbb{R}^2$ is the feature vector for the i^{th} sample
- $\mathbf{x}^{(i)} \in \{0, 1, 2\}$ is the class label for the i^{th} sample

Multinomial Regression (softmax)

Idea: learn a single model with one set of weights per class, all weights are learned simultaneously

$$P(y = k \mid \mathbf{x}) = \frac{\exp[-(\mathbf{w}_k^T \mathbf{x})]}{\sum_j \exp[-(\mathbf{w}_j^T \mathbf{x})]}$$

- Weights: $W \in \mathbb{R}^{3 \times 4}$, where row k is $\mathbf{w}_k^T$
- Bias: $\mathbf{b} \in \mathbb{R}^3$
- $i \in \{1, \dots, 150\}$
- Result: single vector of three class probabilities, summing to 1

Multi-class Classification: One-vs-Rest

Example: Iris Dataset

- $n = 150$ samples
- $d = 4$ features (sepal length/width, petal length/width)
- $K = 3$ classes (setosa, versicolor, virginica)
- $\mathbf{x}^{(i)} \in \mathbb{R}^2$ is the feature vector for the i^{th} sample
- $\mathbf{x}^{(i)} \in \{0, 1, 2\}$ is the class label for the i^{th} sample

One-vs-Rest (sigmoid)

Idea: train K separate binary classifiers, one for each class k , to distinguish class k from all others

$$P(y = k \mid \mathbf{x}^{(i)}) = \frac{1}{1 + \exp[-(\mathbf{w}_k^T \mathbf{x}^{(i)} + b_k)]}$$

- Weights: $\mathbf{w}_k \in \mathbb{R}^4$
- Bias: $b_k \in \mathbb{R}$
- $i \in \{1, \dots, 150\}$
- Result: three scalar probabilities per sample

Exercise: One-vs-Rest Regression

Training

Create new labels

$$y^{(i)} = \begin{cases} 1 & \text{if original class is } k \\ 0 & \text{otherwise} \end{cases}$$

Cross-entropy loss function

$$\mathcal{L}_k = - \sum_{i=1}^n \left[y_k^{(i)} \log(\hat{y}_k^{(i)}) + (1 - y_k^{(i)}) \log(1 - \hat{y}_k^{(i)}) \right]$$

- Typical for classification
- Can be optimized well due to global minimum

Prediction

Predictor for each class k

$$\hat{y}_k = \frac{1}{1 + \exp[-(\mathbf{w}_k^T \mathbf{x} + b_k)]}$$

Select classifier with highest probability

$$\hat{y} = \arg \max_k \hat{y}_k$$

Pandas Methods

`pandas.DataFrame.value_counts()`

- Count the number of occurrences of each unique value in a DataFrame

`pandas.DataFrame.info()`

- Print summary of the DataFrame
- Similar to `pandas.DataFrame.describe()`

`pandas.DataFrame.copy()`

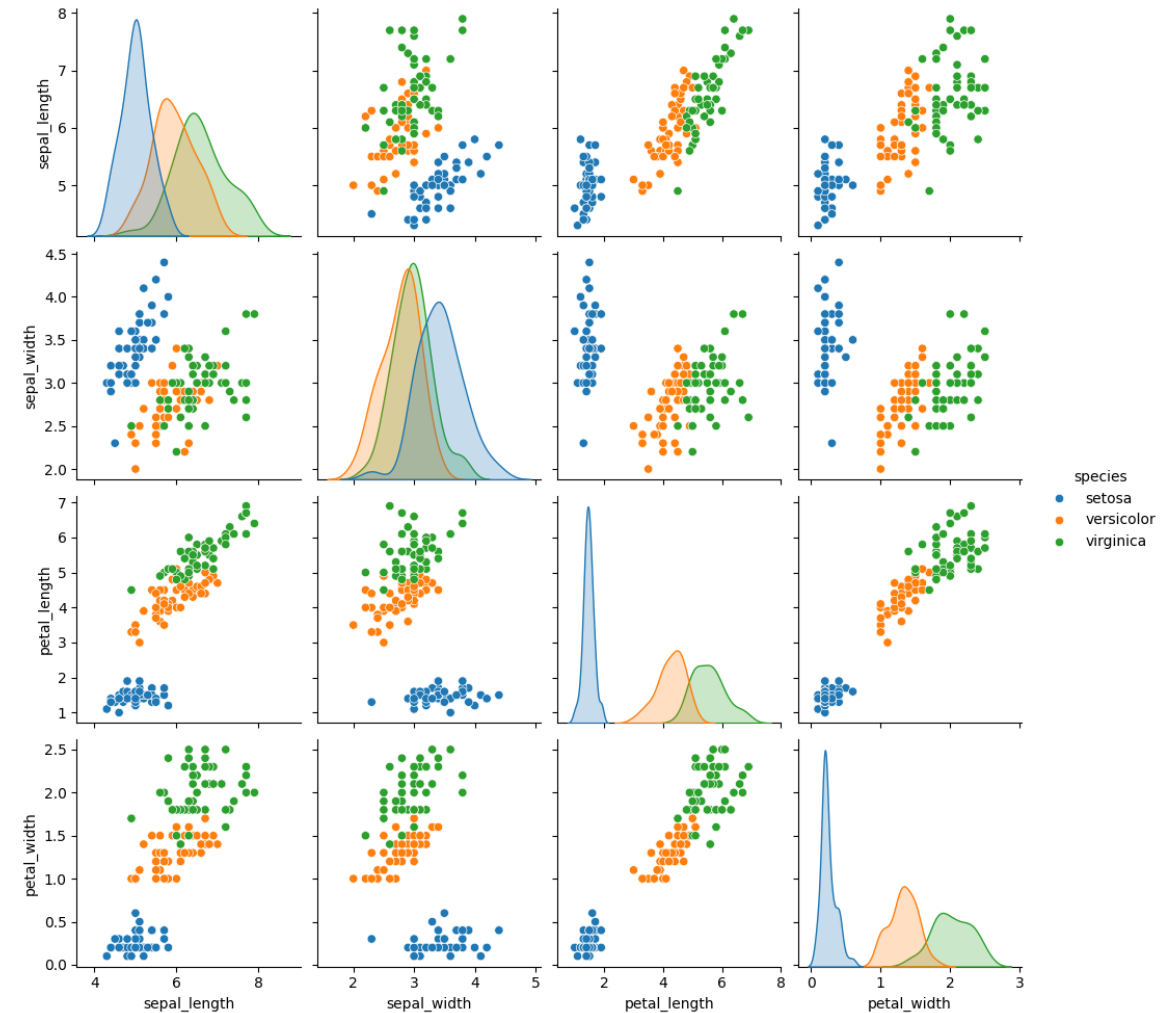
- Copy selected sub-DataFrames (e. g. columns) to a new DataFrame variable
- Changes in the copy will not influence the copy (default behavior)

Pair Plot

`sns.pairplot(DataFrame, hue="labels")`

- Easy way to look at obvious correlations between different features

Logistic Regression
○○○○○



Exercise
●○○○

Scaling Input Data

Scikit-learn StandardScaler

- Scale data to a mean value of $\mu = 0$ and standard deviation of $\sigma = 1$

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- Create a StandardScaler instance:
`scaler = StandardScaler()`
- Compute the mean and standard deviation for each column for the scaling in the next step:
`scaler.fit(training_data)`
- Actual standardization:
`training_data_scaled = scaler.transform(training_data)`

Confusion Matrix

`confusion_matrix(y_true, y_predicted)`

